



Лекция L12

Анализ логов, событий и временной структуры данных

Курс «Машинное обучение и безопасность систем»

Магистратура НИУ ВШЭ (МИЭМ) · ОП «Информационная безопасность и технологии ИИ»

Время изучения ≈ 70 минут

Автор: Силаев Ю. В.



О чём эта лекция за минуту

2 / 52

Часть 1 · Введение

Краткая навигация по теме перед погружением

Логи и события – это не обычная таблица независимых наблюдений. У каждой строки есть момент времени, источник и контекст: она связана с конкретным пользователем, устройством, сессией и правилом, по которому событие вообще возникло. Игнорировать эту структуру – значит решать не ту задачу, что стоит на самом деле.

Время

Порядок и момент события несут смысл. Признаки строятся по окнам, заглядывающим только в прошлое.

Источник

Один поток событий читается на четырёх уровнях: событие, сессия, пользователь, актив.

Проверка

Случайное перемешивание данных разрушает смысл проверки модели. Нужен time-based и group split.

Ключевой тезис: учитывайте время, источник и контекст – иначе проверка модели обманет вас.



Что вы будете уметь после этой лекции

3 / 52

Часть 1 · Введение

Учебные результаты

- | | |
|--|---|
| ✓ вы научитесь описывать лог-событие как объект анализа: время, источник, тип, контекст; | ✓ вы научитесь выбирать time-based или group-based валидацию под событийные данные; |
| ✓ вы научитесь различать event-, session-, user- и asset-level представления одних данных; | ✓ вы научитесь понимать concept drift в данных безопасности и его последствия; |
| ✓ вы научитесь строить базовые временные и оконные признаки (rolling counts, rates, ratios); | ✓ вы научитесь различать точечное событие, последовательность и агрегированное поведение; |
| ✓ вы научитесь объяснять, почему случайный split некорректен для логов; | ✓ вы научитесь объяснять contextual и collective anomalies во временной структуре. |



Что нужно знать до начала

4 / 52

Часть 1 · Введение

Эта лекция опирается на материал предыдущих

L05

Валидация и
утечки

Понятие data leakage и схемы валидации (holdout, CV, time-based, group). Здесь – детальная временная и групповая специфика для логов.

L10

Pipeline

Идея воспроизводимого конвейера и защиты от leakage на этапе fit_transform. Признаки в L12 встраиваются в эту же дисциплину.

L11

Аномалии без
учёта времени

k-means, DBSCAN, Isolation Forest на «снимках» данных; базовая типология аномалий. L12 – развилка: здесь те же данные, но время становится главным.



Семь частей

№	ЧАСТЬ	СЛАЙДЫ	ВРЕМЯ
1	Введение	1-5	≈ 6 мин
2	Мотивация: почему обычный ML ломается на логах	6-9	≈ 5 мин
3	Основное содержание: событие → признак → сессия → валидация → drift	10-37	≈ 38 мин
4	Связки с практикой ИБ: брутфорс, lateral movement, аномалии	38-43	≈ 9 мин
5	Итоги	44-46	≈ 4 мин
6	Источники для углубления	47-48	≈ 2 мин
7	Глоссарий	49-52	≈ 2 мин

Полное время самостоятельного изучения ≈ 66-70 минут.



Завязка: цифра без контекста не значит ничего

Сценарий. Система аутентификации зафиксировала **10 000 событий входа за одну ночь** с одного сервиса. Это много или мало? Инцидент или норма? Без времени, источника и контекста ответа нет – одна и та же цифра поддерживает три несовместимых вывода.

Сбой

Зависший клиент в цикле переподключения генерирует тысячи одинаковых событий с одного IP. Поведение машинное, не человеческое.

Контекст: источник

Брутфорс

Распределённый перебор паролей: много уникальных учётков, высокая доля fail, всплеск строго ночью.

Контекст: время + ratio

Норма

Плановый ночной батч сервис-аккаунта по расписанию. Так бывает каждую ночь в это же окно.

Контекст: профиль по времени



Почему обычный ML здесь ломается

7 / 52

Часть 2 · Мотивация

Три молчаливых допущения табличного ML – и как логи их нарушают

Стандартный табличный ML опирается на три допущения о данных. На логах каждое из них ложно – и именно поэтому привычные приёмы дают неверный результат.

Независимость

Допущение ML

Строки независимы друг от друга.

На логах: События одного пользователя или сессии связаны: одно следует из другого. Это зависимые наблюдения.

Обменимость

Допущение ML

Порядок строк не важен, их можно перемешать.

На логах: Порядок несёт смысл: вход → повышение прав → доступ к данным – это атака; обратный порядок – нет.

Стационарность

Допущение ML

Распределение со временем не меняется.

На логах: «Нормальное» поведение дрейфует: новый сервис, релиз, смена графика работы меняют базовую линию.



Что происходит, когда время игнорируют при проверке модели

ИБ-КЕЙС

Детектор brutфорса с random split

Инженер обучает модель обнаружения brutфорса на логах входа. Данные он делит **случайным образом** (train_test_split, shuffle=True). На тесте модель показывает **ROC-AUC 0.99**. Отчёт выглядит блестяще – модель уходит в прод.

В проде: качество рушится. Атаки нового дня не ловятся, алертов либо лавина, либо тишина. Метрика 0.99 была фикцией.

Почему так вышло. Перемешивание разбросало события одной атаки и одной ночи и в train, и в test. Модель «подглядела» будущее: на экзамене ей достались ответы из того же инцидента, что и в учебнике. Это и есть утечка по времени – разберём её механику в Части 3.



Узнаваемый пример, который сопровождает вас от L02 до конца курса

CIC-IDS-2018 · ЧТО ЭТО

~16 млн	flow-записей сетевого трафика
10 дней	период сбора — это и есть временная ось
80	признаков на каждую flow-запись
14 + 1	типов атак плюс класс Benign

Почему в этой лекции он — временной

В L11 этот же датасет рассматривался как **таблица объектов** — снимок без времени. **Здесь он впервые читается как поток во времени:**

- 10 дней дают естественную границу train / test по времени;
- оконные признаки агрегируются только по прошлым flow;
- атаки появляются в конкретные дни — порядок имеет значение.

При следующих упоминаниях в лекции — просто «CIC-IDS-2018», без повторной карточки.



БЛОК 1

Что такое лог-событие

01

Сейчас разберём анатомию отдельного события и переход от сырых записей к агрегированным представлениям – фундамент, на котором строятся все признаки лекции.



Определение: лог-событие

11 / 52

Часть 3 · Блок 1

Семь полей, без которых событие нельзя анализировать

ОПРЕДЕЛЕНИЕ

Лог-событие — это запись об одном факте в системе, описываемая набором полей: **когда, откуда, что произошло, с кем, над чем, с каким исходом и насколько серьёзно.**

timestamp

когда — момент времени

source

откуда — хост, сервис, сенсор

event type

что — тип события (login, access)

subject

кто — пользователь, процесс, IP

object

над чем — файл, ресурс, учётка

outcome

исход — success / failure / denied

severity

серьёзность — info / warn / critical



Анатомия события на примере

12 / 52

Часть 3 · Блок 1

Одна строка лога входа, разобранная по семи полям

```
2018-02-21 03:14:07 auth-srv-02 user_login jdoe /vpn/gw FAILURE warning
```

ПОЛЕ	ЗНАЧЕНИЕ	ЧТО ЭТО ГОВОРIT АНАЛИТИКУ
timestamp	2018-02-21 03:14:07	ночь, 03:14 – нетипичное время для этого пользователя
source	auth-srv-02	сервер аутентификации №2 – где зафиксировано событие
event type	user_login	попытка интерактивного входа
subject	jdoe	учётная запись, от имени которой идёт вход
object	/vpn/gw	ресурс назначения – VPN-шлюз
outcome	FAILURE	вход не удался – кандидат в признак брутфорса
severity	warning	уровень, присвоенный источником события



Сырое событие vs агрегированное представление

13 / 52

Часть 3 · Блок 1

Одно событие почти бесполезно – смысл рождается в агрегации

Отдельное лог-событие редко отвечает на вопрос «атака или норма». Один FAILURE входа – это опечатка пользователя. Но **50 FAILURE за минуту с одного источника по разным учёткам** – это уже сигнал. Смысл появляется, когда мы агрегируем события по сущности и по временному окну.

СЫРЫЕ СОБЫТИЯ

```
03:14:07 jdoe FAILURE
03:14:09 asmith FAILURE
03:14:11 jdoe FAILURE
03:14:13 root FAILURE
... (ещё 46 строк)
```



агрегация
по окну 1 мин

АГРЕГИРОВАННЫЙ ОБЪЕКТ

```
window          = 03:14-03:15
failed_logins    = 50
unique_users     = 12
success          = 0
source           = auth-srv-02
```

Объектом анализа становится не строка лога, а агрегат «сущность × окно». Именно у него есть признаки, которые что-то значат.



Что мы только что разобрали

1

Семь полей

Лог-событие описывается набором: timestamp, source, event type, subject, object, outcome, severity.

2

Событие ≠ независимая точка

Одна строка лога почти бесполезна для решения «атака или норма» – она получает смысл только в контексте.

3

Агрегация рождает признак

Объект анализа – это агрегат «сущность × временное окно», а не отдельная запись. На нём и строятся признаки.



БЛОК 2

От события к признаку

Уровни анализа, временные и оконные признаки. Как один и тот же поток событий превращается в матрицу «объект × признак» — и почему уровень анализа выбирают до построения признаков.

0

2



Уровни анализа: событие, сессия, пользователь, актив

16 / 52

Часть 3 · Блок 2

Один поток событий можно собрать в объекты разного масштаба

ОПРЕДЕЛЕНИЕ

Уровень анализа — это сущность, относительно которой агрегируются события и формируется один объект для модели. Один и тот же лог даёт разные объекты на разных уровнях.

event-level

Объект = одно событие

Классификация отдельной записи: этот вход подозрителен?

session-level

Объект = одна сессия

Поведение в рамках сеанса: эта сессия аномальна?

user-level

Объект = профиль пользователя за окно

Отклонение от привычного профиля: этот юзер ведёт себя не как обычно?

asset-level

Объект = устройство / сервис / IP

Состояние актива: на этом хосте всплеск ошибок?



Один поток событий – четыре таблицы

17 / 52

Часть 3 · Блок 2

Выбор уровня меняет, что является строкой, а что – признаком

Уровень анализа выбирают **до** построения признаков: он определяет, что считать одним объектом (строкой матрицы). Из одного лога входа получаются четыре совершенно разные обучающие выборки.

УРОВЕНЬ	СТРОКА = ОБЪЕКТ	ПРИМЕРЫ ПРИЗНАКОВ	СКОЛЬКО ОБЪЕКТОВ
event-level	одна строка лога	outcome, час, тип события	<i>≈ N строк (по числу событий)</i>
session-level	одна сессия входа	длительность, число попыток, доля fail	<i>меньше – события схлопнуты в сессии</i>
user-level	пользователь за окно (час/день)	событий за окно, уник. хостов, ночная доля	<i>по числу активных пользователей</i>
asset-level	хост / сервис за окно	ошибок за окно, уник. источников, rate	<i>по числу активов</i>

Та же задача, но на разных уровнях – это разные модели с разной валидацией. Об этом – в блоке про split.



Сам момент события – уже информация

Из одного только timestamp извлекается семейство признаков. В ИБ они работают потому, что «нормальное» поведение привязано ко времени суток, дню недели и ритму инфраструктуры.

час суток

0-23

*ночной вход вне графика
пользователя – слабый
сигнал*

день недели

пн-вс

*активность в выходные для
рабочей учётки
подозрительна*

время с последнего события

Δt , секунды

*серия с $\Delta t \approx 0$ – машинная, не
человеческая*

частота событий

событий в минуту

*всплеск частоты – кандидат
в брутфорс / сканирование*

Важно: час суток – циклический признак (23 и 0 рядом). Кодировать его как $\sin/\cos$, а не как число 0-23, иначе модель решит, что полночь «далеко» от 23:00.



Оконные признаки: rolling counts, rates, ratios

19 / 52

Часть 3 · Блок 2

Агрегация по скользящему окну, заканчивающемуся в момент события

$$\text{count_W}(t) = | \{ e : t - W \leq \text{time}(e) < t \} |$$

число прошлых событий сущности в окне: левая граница включается, момент t – нет

Компоненты

t – момент текущего события (правая граница окна);

W – длина окна (60 с, 5 мин, 1 ч, 24 ч);

$< t$ – строгое неравенство: текущее и будущее НЕ входят.

Семейство оконных признаков

count	число событий за окно
unique count	число уникальных значений (юзеров, IP)
rate	count / длительность окна
ratio	доля, напр. fail / (fail + success)



Пример: признаки для брутфорса по числам

20 / 52

Часть 3 · Блок 2

Окно 5 минут на user-level, источник auth-srv-02

ИБ-КЕЙС

Считаем оконные признаки для учётки jdoe на момент $t = 03:15:00$, окно $W = 5$ мин ($03:10-03:15$).

ПРИЗНАК	ЗНАЧ.	КАК СЧИТАЕТСЯ	ИНТЕРПРЕТАЦИЯ
failed_logins_5m	47	count FAILURE за окно	очень высоко для одной учётки
success_logins_5m	0	count SUCCESS за окно	ни одного успеха – перебор
fail_ratio_5m	1.00	fail / (fail + success)	100% неудач
unique_src_ip_5m	9	unique count IP-источников	распределённый источник
mean_dt_sec	6.4	среднее Δt между попытками	машинный ритм, не человек

Ни один признак сам по себе не приговор – но вместе они дают сильный, объяснимый сигнал брутфорса.



Что мы только что разобрали

1

Уровень определяет объект

event / session / user / asset – выбор уровня задаёт, что является строкой обучающей выборки. Уровень выбирают ДО признаков.

2

Время – это признак

Из timestamp растут час, день недели, Δt , частота. Циклические признаки кодируют через $\sin/\cos$.

3

Окно всегда смотрит назад

Оконные count / unique / rate / ratio агрегируются строго по прошлым событиям: граница $< t$, а не $\leq t$.



БЛОК 3

Сессии и порядок событий

Группировка событий в сессии и почему порядок несёт смысл. Различаем три уровня сигнала: одно событие, последовательность событий и агрегированное поведение.

0

3



Техника, которая может быть новой – разберём подробно

ОПРЕДЕЛЕНИЕ

Sessionization (сессионизация) – группировка последовательных событий одной сущности в **сессии**: события одного и того же субъекта объединяются в один сеанс, пока паузы между ними не превышают порог бездействия θ (**session timeout**).

Зачем это нужно

Сессия – естественная единица поведения. Отдельное событие говорит мало, а сессия – это законченный сеанс работы пользователя или процесса: вход, серия действий, выход. На уровне сессии появляются признаки, недоступные на уровне события: длительность, число и порядок шагов, доля неудач внутри сеанса, переходы между ресурсами.

Типичный θ для веб-сессий – 30 минут; для машинных процессов окно подбирают под ритм событий.



Как нарезать поток на сессии

24 / 52

Часть 3 · Блок 3

Алгоритм в три шага и числовой пример при $\vartheta = 30$ минут

Алгоритм

1

Сортировка

События одной сущности упорядочиваются строго по времени.

2

Разрыв по простоему

Если пауза до следующего события $> \vartheta$ — начинается новая сессия.

3

Агрегация по сессии

Внутри каждой сессии считаются признаки: длительность, число шагов, доля fail.

Пример: события jdoe

09:01 login

09:04 open /report

09:07 logout

— *пауза 2 ч 53 мин $> \vartheta$* —

12:00 login

12:02 open /vault

12:03 logout

Сессия A = 3 события, 6 мин. **Сессия B** = 3 события, 3 мин.

Один поток из 6 событий превратился в 2 объекта-сессии — это и есть *session-level* представление.



Точка, последовательность, агрегированное поведение

25 / 52

Часть 3 · Блок 3

Три уровня сигнала – три разных способа поймать атаку

Точечное событие

point anomaly

Один факт сам по себе аномален. *Вход root из внешней сети в 03:00.*

Последовательность

последовательность

Каждое событие нормально, но их порядок – нет. *login → повышение прав → доступ к чужому ресурсу → выгрузка. Это lateral movement.*

Агрегированное поведение

collective anomaly

Профиль за окно отклоняется от привычного. *Юзер, обычно делающий 20 действий в день, сделал 2000.*

Исторически **последовательности** моделировали скрытыми марковскими моделями (НММ); сегодня чаще берут признаки переходов и *n*-граммы событий. Глубокие модели последовательностей – вне рамок этой лекции.



Что мы только что разобрали

1

Сессия – единица поведения

Sessionization группирует события одной сущности по таймауту θ . Сессия даёт признаки, недоступные на уровне события.

2

Порядок – это сигнал

Те же события в другом порядке означают другое. login → escalation → exfiltration – атака; иной порядок – нет.

3

Цепочка слабых сигналов = инцидент

Атаку часто не видно в одном событии. Она проявляется как последовательность или как отклонение профиля за окно.



БЛОК 4

Как проверять модель на логгах

Здесь живёт главный тезис лекции: **случайное перемешивание временных данных разрушает смысл проверки**. Разбираем, почему так, и как делать правильно: time-based split, rolling validation, group split.





Почему случайный split лжёт

28 / 52

Часть 3 · Блок 4

Возвращаемся к детектору с ROC-AUC 0.99 из части «Мотивация»

ЧАСТАЯ ОШИБКА

«Раз данных много, перемешаю строки и поделю 80/20 – как всегда.

`train_test_split(shuffle=True)` – и метрика честная.»

ПОЧЕМУ ЭТО НЕВЕРНО

Перемешивание разбрасывает события одной атаки и одной ночи и в train, и в test. Модель видит на обучении соседние события того же инцидента, что попали в тест, – и «узнаёт» их по почти одинаковым признакам. Это утечка по времени: будущее просочилось в прошлое. На тесте – ROC-AUC 0.99, в проде – провал, потому что настоящие будущие атаки таких подсказок не дают.

Правило: на временных данных тест должен лежать в будущем относительно train. Всегда.



Определение: time-based split

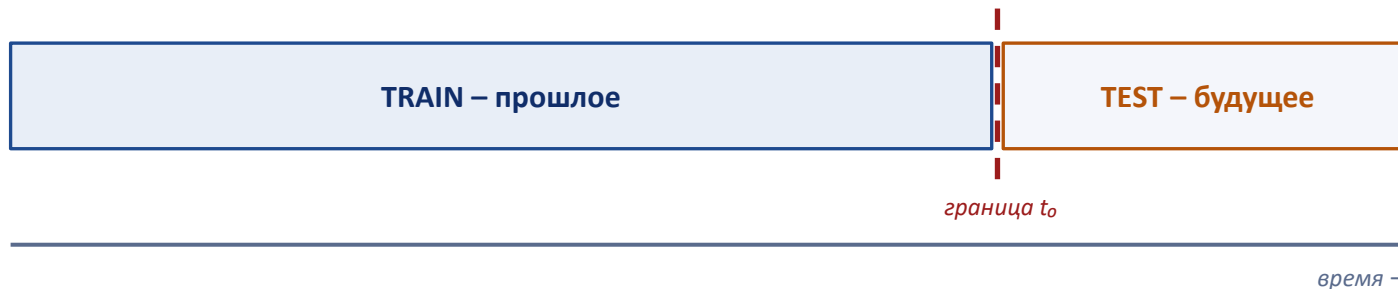
29 / 52

Часть 3 · Блок 4

Общая идея – из L05; здесь – детально для логов

ОПРЕДЕЛЕНИЕ

Time-based split – разбиение по времени: всё, что произошло **до** момента-границы, идёт в train; всё **после** – в test. Модель обучается на прошлом и проверяется на будущем, как в эксплуатации.



Граница t_0 выбирается так, чтобы test покрывал реалистичный «следующий» период эксплуатации. Никакое событие из test не должно влиять на признаки в train.



Rolling и expanding validation

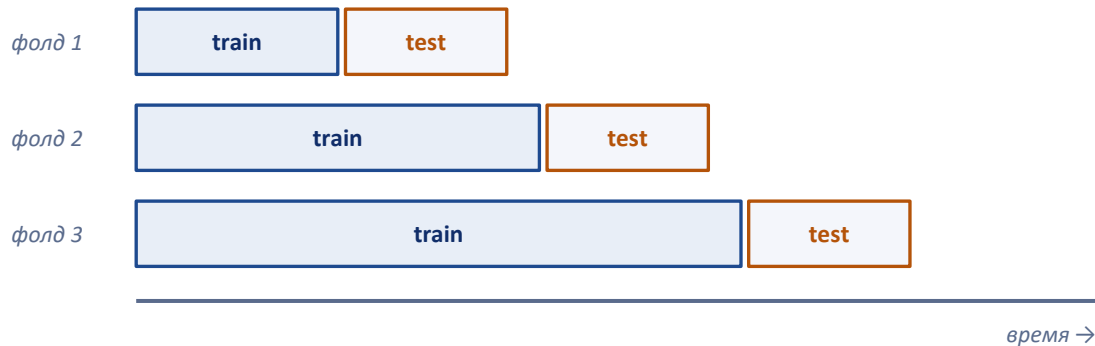
30 / 52

Часть 3 · Блок 4

Несколько временных фолдов вместо одной границы

Одна граница даёт одну оценку. Чтобы оценка была устойчивой, тест прокатывают по времени несколькими фолдами – train всегда строго раньше своего test.

Expanding window



Rolling

train – окно фикс. длины, скользит вперёд.

Expanding

train растёт, накапливая историю.

Итоговая метрика – среднее по фолдам. В каждом фолде train целиком раньше test: правило времени не нарушается ни разу.



Group split: события сущности не делят

31 / 52

Часть 3 · Блок 4

Групповая специфика *split* – линия L05 → L10, здесь для логов

ОПРЕДЕЛЕНИЕ

Group split – разбиение, при котором все события одной группы (пользователя, устройства, организации, IP) целиком попадают либо в train, либо в test, но никогда не делятся между ними.

БЕЗ group split

События пользователя jdoe есть и в train, и в test.
Модель запоминает индивидуальный стиль jdoe, а не обобщает. На тесте – высокая метрика; на новом пользователе – провал. Это утечка по группе.

С group split

jdoe целиком в train ИЛИ целиком в test. Тест содержит только пользователей, которых модель не видела. Метрика отражает обобщение на новых субъектов – то, что и нужно в проде.

Часто нужны оба сразу: время + группа. В *sklearn* – *GroupKFold* и *TimeSeriesSplit*; детали реализации – в L10.



Самый коварный *leakage*: он прячется внутри признака

ЧАСТАЯ ОШИБКА

Признак считают по окну, центрированному на событии, или по всей истории включая будущее: `rolling(window).mean()` без сдвига, или агрегат «за весь день», когда решение принимается в полдень.

ПОЧЕМУ ЭТО НЕВЕРНО

Признак «заглянул» за момент решения t . В обучении он содержит информацию о событиях, которых в реальном времени ещё не было. Метрика на валидации завышена, а в эксплуатации признак физически невозможно посчитать так же – модель деградирует. Граница окна должна быть строго $< t$ (см. блок про признаки).

Проверка: «мог ли я посчитать этот признак в момент t , не зная будущего?» Если нет – это утечка.



10 дней как готовая временная ось для валидации

ИБ - кейс

В CIC-IDS-2018 атаки распределены по 10 дням. Естественная граница – по дням: ранние дни в train, поздние в test.



TRAIN: дни 1-7

TEST: 8-10

Что это даёт. Test содержит дни, которых модель не видела, – как новые сутки в SOC. Оконные признаки внутри каждого дня агрегируются только по прошлым flow. Так метрика отражает реальную способность ловить завтрашние атаки, а не вчерашние.



Что мы только что разобрали

1

Время режет split

Тест всегда в будущем относительно train. Time-based split и rolling/expanding validation вместо random.

2

Группа режет split

События одной сущности не делятся между train и test, иначе модель запоминает субъекта вместо обобщения.

3

Окно смотрит назад

Оконные признаки агрегируются строго по прошлому ($< t$). Иначе будущее протекает в обучение.

4

Random split = ложь

На временных данных перемешивание даёт завышенную метрику и провал в проде. Это и есть главный тезис лекции.



БЛОК 5

Когда нормальное меняется

Две концепции, которые делают валидацию на логах ещё тоньше: concept drift (нормальное поведение дрейфует во времени) и label delay (истинная разметка приходит с задержкой).

0

5



Введение здесь – развитие в L17

ОПРЕДЕЛЕНИЕ

Concept drift – изменение со временем зависимости между признаками и целевой переменной (или самого распределения данных). То, что было «нормальным» вчера, перестаёт быть нормальным сегодня.

Почему это критично именно в ИБ

Среда меняется

новый сервис, релиз, смена графика работы сдвигают базовую линию «нормы».

Противник адаптируется

атакующий меняет тактику, чтобы не попадать под старые сигнатуры поведения.

Профиль пользователей

новые роли, отделы, устройства приходят в систему и меняют типичное поведение.

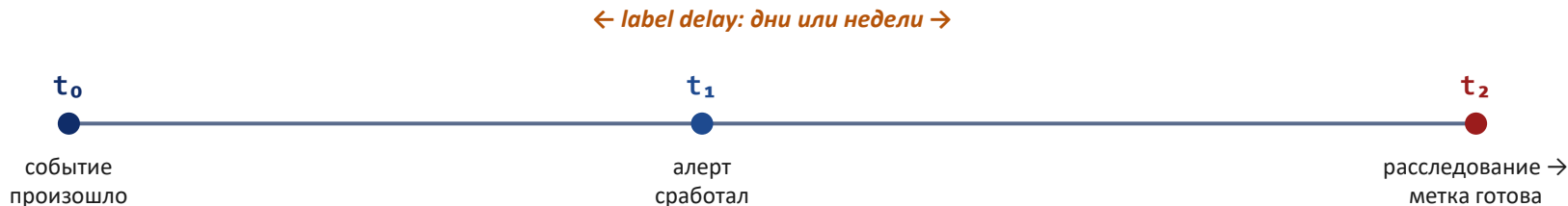
Следствие: модель, отличная на старом тесте, тихо деградирует в проде. Поэтому её переобучают и мониторят. Полная типология drift и меры борьбы – в L17.



Задержка разметки – термин, который знают немногие

ОПРЕДЕЛЕНИЕ

Label delay (задержка разметки) – разрыв во времени между тем, как событие произошло, и тем, как для него становится известна истинная метка. В ИБ метку «инцидент / не инцидент» часто ставят только после расследования.



Что это меняет на практике

- **Свежие данные ещё не размечены**
– оценивать модель «прямо сейчас» не на чем; нужна выдержка времени.
- **Признаки после расследования – это утечка**
– нельзя кормить модель тем, что аналитик узнал уже постфактум.



ЧАСТЬ 4

Логи в реальных задачах ИБ

Теория разобрана – теперь два развёрнутых кейса с числами, типология аномалий во времени, роль профиля нормы и ключевое различие трёх задач: обнаружение, прогнозирование, ретроспектива.



Кейс 1: bruteforce по временным окнам

39 / 52

Часть 4 · Связки с ИБ

Полный путь от события до решения – на user-level, окно 5 минут

ИБ-КЕЙС

Задача: на потоке событий входа отметить учётки, по которым идёт перебор паролей, в почти реальном времени.

1 · Признаки

failed_5m, fail_ratio_5m,
unique_src_ip_5m, mean_dt_sec –
оконные, граница < t.

2 · Модель + порог

Бустинг или логрег. Порог под цену
ошибки (линия L04 → L07): пропуск
дороже ложной тревоги.

3 · Валидация

Time-based split по дням + group по
учёткам. Метрика – precision@top-k
алертов.

РЕЗУЛЬТАТ

На учётке **jdoe**: failed_5m=47, fail_ratio=1.00, unique_ip=9 → алерт с высокой уверенностью. *Объяснимо: каждый признак виден аналитику.*



Кейс 2: lateral movement как последовательность

40 / 52

Часть 4 · Связки с ИБ

Когда каждое событие нормально, а опасна их цепочка

ИБ-КЕЙС

Атакующий с одной скомпрометированной учётки постепенно расширяет доступ по сети. Каждый шаг по отдельности – легитимное действие.

ВХОД

host-A, рабочее время – норма



повышение прав

локальный админ – бывает



доступ к host-B

новый для этой учётки хост



выгрузка

большой объём на внешний адрес

Точечный детектор молчит. Каждое событие проходит как норма – порог не сработает ни на одном.

Сессия / последовательность ловит. Признаки переходов и порядок шагов выдают цепочку как единый паттерн.



Contextual и collective anomalies во времени

Базовая типология – в L11; здесь – её временное проявление

Во временных данных два типа аномалий проявляются особым образом. Значение, нормальное в одном контексте, становится аномальным в другом – а отдельные нормальные события вместе образуют аномальную группу.

CONTEXTUAL ANOMALY

Аномалия в контексте

Значение само по себе нормально, но не для этого времени/субъекта. Контекст здесь – время суток, день недели, роль.

Пример: вход `jdoe` в 03:14 – норма для ночной смены, аномалия для офисного работника 9-18.

COLLECTIVE ANOMALY

Аномалия группы

Отдельные события нормальны, но их последовательность или всплеск – нет. Аномалия живёт на уровне группы во времени.

Пример: 50 одиночных `FAILURE` за минуту – каждый нормален, всплеск из них – брутфорс.



Профиль нормы зависит от времени и роли

42 / 52

Часть 4 · Связки с ИБ

«Нормальное» – не одно на всех: оно условное

Единого порога «нормы» не существует. То, что подозрительно для одного субъекта в одно время, – рутина для другого. Профиль нормы строят по сегментам.

Время суток

Ночная активность аномальна для дневной смены и нормальна для ночной.

Роль / отдел

Админ ходит по многим хостам штатно; бухгалтер – нет.

Тип актива

Сервер БД и ноутбук стажёра имеют разные нормальные паттерны.

Сезонность

Конец квартала, релизы, обслуживание дают законные всплески.

ПРАКТИКА СОС

Группировка повторяющихся алертов по профилю снижает шум: однотипные срабатывания одного актива в одно окно схлопывают в один инцидент, а не заваливают аналитика.



Обнаружение $\neq$ прогноз $\neq$ ретроспектива

43 / 52

Часть 4 · Связки с ИБ

Три разные задачи, которые студенты путают

ЧАСТАЯ ОШИБКА

Считать, что «найти атаку» – одна задача. На деле это три задачи с разной допустимостью будущей информации и разной валидацией.

Обнаружение

в момент t

Решение принимается сейчас, по прошлым данным. Будущее недоступно в принципе.

Прогнозирование

до события

Предсказать, что атака произойдёт. Самая строгая по утечке: ни одного признака из будущего.

Ретроспектива

после, оффлайн

Разбор инцидента задним числом. Будущее доступно – но это НЕ модель для прода.

Главное: ретроспективный анализ может «знать» будущее, а обнаружение и прогноз – нет. Перенос признаков из ретроспективы в прод – это утечка.



Семь утверждений, которые стоит унести

- 1 Логи – не таблица независимых строк: у события есть время, источник и контекст.
- 2 Уровень анализа (событие / сессия / пользователь / актив) выбирают до построения признаков.
- 3 Оконные признаки агрегируются строго по прошлому: граница окна $< t$.
- 4 На временных данных тест всегда в будущем относительно train – time-based split.
- 5 События одной сущности не делят между train и test – group split.
- 6 Concept drift и label delay делают «честную» оценку ещё тоньше.
- 7 Инцидент – это часто цепочка слабых сигналов, а не одно событие.



Где применять, где не применять

45 / 52

Часть 5 · Итоги

Границы темы L12 – чтобы не выйти за рамки

ЭТО – ТЕРРИТОРИЯ L12

- ✓ Признаки событий и сессий из логов;
- ✓ оконные агрегаты с границей $< t$;
- ✓ time-based и group валидация;
- ✓ sessionization и анализ порядка;
- ✓ распознавание contextual / collective аномалий во времени.

ЭТО – ВНЕ ЭТОЙ ЛЕКЦИИ

- ✗ глубокая теория временных рядов;
- ✗ ARIMA, ETS, state-space модели;
- ✗ LSTM / GRU для логов, глубокие seq-модели;
- ✗ потоковая обработка и real-time архитектуры;
- ✗ корреляционные правила SIEM как дисциплина.



Куда ведут линии этой лекции в остальном курсе

L13

Основы нейросетей

Следующая лекция: переход от признакового ML к нейросетям.

L15

Тексты и журналы событий

Лог-сообщения как тексты: парсинг, TF-IDF, эмбединги.

L17

Устойчивость к drift

Полная типология concept drift и меры борьбы с ним.

L20

Приватность в логах

Запоминание данных и приватность применительно к логам.



Минимум, который нужно прочитать по теме лекции

1

Hastie, Tibshirani, Friedman. *The Elements of Statistical Learning.*

Гл. 7 «Model Assessment and Selection» – оценка качества и выбор модели, основа корректной валидации.

2

Hastie, Tibshirani, Friedman. *The Elements of Statistical Learning.*

Гл. 14 «Unsupervised Learning» – типология аномалий (база, развёрнута в L11).

3

Gama, Žliobaitė, Bifet et al.. *A Survey on Concept Drift Adaptation (ACM CSUR, 2014).*

Обзор concept drift и подходов к адаптации – введение к теме, развитие в L17.



Для тех, кто хочет глубже, и практические материалы

ДОПОЛНИТЕЛЬНО

- Chandola, Banerjee, Kumar. Anomaly Detection: A Survey (ACM CSUR, 2009) – типы аномалий, в т. ч. contextual и collective.
- Aggarwal. Outlier Analysis – гл. о временных и последовательных выбросах.
- Bifet et al. Machine Learning for Data Streams – потоковые данные и drift.

ПРАКТИКА

- sklearn: TimeSeriesSplit и GroupKFold – документация `model_selection`.
- pandas: `rolling()` и `groupby().rolling()` – оконные агрегаты с правильной границей.
- CIC-IDS-2018 – датасет на сайте Canadian Institute for Cybersecurity (UNB).



Ключевые термины лекции – для быстрой справки

ТЕРМИН	ОПРЕДЕЛЕНИЕ
лог-событие	Запись о факте в системе: timestamp, source, event type, subject, object, outcome, severity.
уровень анализа	Сущность, по которой агрегируются события: event / session / user / asset-level.
оконный признак	Агрегат (count, rate, ratio) по скользящему окну, заканчивающемуся строго до момента t .
sessionization	Группировка событий одной сущности в сессии по таймауту бездействия θ .
time-based split	Разбиение по времени: train – прошлое, test – будущее относительно границы.
group split	Разбиение, при котором все события одной сущности целиком в train или в test.



Ключевые термины лекции – для быстрой справки

ТЕРМИН	ОПРЕДЕЛЕНИЕ
rolling / expanding validation	Несколько временных фолдов: train всегда раньше своего test; метрика – среднее по фолдам.
concept drift	Изменение со временем зависимости признаки → цель или распределения данных.
label delay	Задержка между событием и получением его истинной метки (часто после расследования).
contextual anomaly	Значение нормально само по себе, но аномально в данном контексте (время, роль).
collective anomaly	Отдельные события нормальны, но их группа или последовательность – аномальна.
lateral movement	Постепенное расширение доступа атакующим по сети как последовательность шагов.



Карта лекции одним взглядом

51 / 52

Часть 7 · Глоссарий

Пять блоков основного содержания и их связь



Сквозная нить: время – главное действующее лицо. От того, как устроено событие, до того, как мы честно проверяем модель на нём.



Конец лекции L12

Анализ логов, событий и временной структуры данных

Дальше в курсе: L13 – основы нейросетей.

Курс «Машинное обучение и безопасность систем» · НИУ ВШЭ (МИЭМ) · Силаев Ю. В.